

Week 8 - SQL for Data Analysis

Celebal Summer Internship 2026

Submitted By

Name: Akash Patra

Phase 1 – Introduction and Project Overview

Introduction

Data has become one of the most valuable assets for modern businesses. Every online platform continuously collects information related to customers, products, orders, and transactions. However, raw data alone cannot support business decisions unless it is properly organized, cleaned, stored, and analyzed.

The objective of this project is to demonstrate a complete end-to-end data analysis workflow using Python, SQLite, and SQL. Instead of using a publicly available dataset, realistic e-commerce data was generated programmatically. The generated data was then cleaned, validated, stored inside a relational database, and analyzed using SQL queries to extract meaningful business insights.

The project simulates a real-world e-commerce environment and demonstrates how data moves through different stages, starting from raw data generation to final business analysis.

Project Overview

This project was developed as part of the **Celebal Summer Internship 2026 – Week 8 Assignment**.

The project follows a structured workflow consisting of multiple phases. First, realistic datasets are generated using Python. These datasets intentionally contain several data quality issues to simulate real-world business scenarios.

In the next phase, the generated datasets are cleaned and validated to improve data quality. After cleaning, the datasets are loaded into a SQLite database where multiple SQL queries are executed to perform business analysis.

Finally, Python is integrated with SQLite to demonstrate how SQL queries can be executed directly from a Python application.

This project provides practical exposure to data generation, preprocessing, relational databases, SQL programming, and Python-SQL integration.

Project Objectives

The main objectives of this project are:

- Generate realistic e-commerce datasets using Python.
 - Simulate customer, product, and order-related information.
 - Introduce realistic data quality issues for cleaning practice.
 - Perform data cleaning and validation.
 - Store cleaned data inside a SQLite relational database.
 - Perform SQL-based business analysis.
 - Apply advanced SQL concepts such as Window Functions and Common Table Expressions (CTEs).
 - Perform Customer Segmentation and Cohort Analysis.
 - Integrate Python with SQLite for automated SQL execution.
 - Gain practical experience in solving business problems using data.
-

Project Workflow

The complete project is divided into the following phases.

Phase 1 – Data Generation

Realistic e-commerce datasets were generated using Python. Customer information was generated using the Faker library, while transactional data such as products, orders, discounts, and quantities were created using Python's random module.

Phase 2 – Data Cleaning

The generated datasets were cleaned by correcting incorrect date formats, validating email addresses, handling missing values, standardizing product names, and verifying referential integrity.

Phase 3 – SQLite Database Creation

The cleaned datasets were loaded into a SQLite database. Separate relational tables were created for customers, products, orders, and order items.

Phase 4 – SQL Analysis

SQL queries were executed to perform business analysis. The implementation includes basic SQL queries, intermediate queries, advanced analytical queries, Window Functions, Common Table Expressions (CTEs), Customer Segmentation, and Cohort Analysis.

Phase 5 – Python and SQLite Integration

Python was connected with SQLite to execute SQL queries programmatically and display the results using Pandas DataFrames.

Technologies Used

The following technologies were used throughout the project.

Technology	Purpose
Python	Data generation, data cleaning, database operations, and automation
Faker	Generating realistic synthetic customer information
Pandas	Reading, cleaning, processing, and exporting datasets
SQLite	Relational database for storing structured data
sqlite3	Connecting Python with SQLite
SQL	Performing business analysis on relational data
Visual Studio Code	Writing and executing Python and SQL code
Jupyter Notebook	Preparing project documentation and presentation

Why These Technologies Were Used?

Each technology used in this project serves a specific purpose.

Python

Python was selected because it provides a simple and efficient way to automate data generation, data cleaning, database creation, and SQL execution.

It also supports a wide range of libraries that simplify data analysis tasks.

Faker

The Faker library was used to generate realistic customer information such as names, email addresses, and registration dates.

Using Faker helps create synthetic data that closely resembles real-world customer records without exposing any personal information.

Pandas

Pandas was used for reading CSV files, cleaning datasets, updating incorrect values, validating records, and exporting cleaned datasets.

Its DataFrame structure makes working with tabular data much easier and more efficient.

SQLite

SQLite was selected because it is a lightweight relational database that does not require a separate database server.

It stores the complete database inside a single file, making it ideal for academic projects and small-scale applications.

SQL

SQL was used to retrieve, analyze, and summarize data stored in the database.

Using SQL makes it possible to generate meaningful business insights from structured datasets quickly and efficiently.

Project Structure

The project is organized into multiple folders, where each folder is responsible for a specific phase of the implementation.

```
Week-8/
|
├── data/
│   ├── raw/
│   └── cleaned/
├── database/
│   └── ecommerce.db
├── python/
│   └── generate_data.py
```

```
|
|   ├── clean_data.py
|   ├── database.py
|   └── run_queries.py
|
|── sql/
|   └── basic_queries.sql
|
|── screenshots/
|   ├── data_generation/
|   ├── data_cleaning/
|   ├── database/
|   ├── integration/
|   ├── sql_queries/
|   └── final_output/
|
|── report/
|   └── notebook.ipynb
|
|── README.md
```

Observation

This phase provides an overview of the complete project and explains the overall workflow followed during implementation.

The project combines Python, SQLite, and SQL to create a complete data analysis pipeline that begins with data generation and ends with business analysis and automated SQL execution.

The following phases explain each implementation step in detail, including the source code used, the purpose of each phase, the output generated, and the observations made during execution.

Phase 2 – Data Generation

Purpose

The first implementation phase of this project is data generation. Since no real e-commerce dataset was provided for this assignment, a realistic synthetic dataset was created using Python.

The objective of this phase is to simulate the working of an online shopping platform by generating customer records, product details, order information, and transaction records. These datasets act as the foundation for all the remaining phases of the project, including data cleaning, database creation, SQL analysis, and Python-SQL integration.

Automating the data generation process makes the project scalable, reduces manual effort, and produces consistent data that can be reused for testing and analysis.

Source Code

The complete implementation of the data generation phase is available in the following file:

```
Week-8/  
└─ python/  
    └─ generate_data.py
```

This Python program automatically generates all the required datasets and stores them inside the **data/raw** directory.

Implementation

The data generation program was developed using Python and several supporting libraries.

The program performs the following tasks:

- Generates customer records.
- Generates product information.
- Creates customer orders.
- Creates order item details.
- Introduces intentional data quality issues.
- Stores all generated datasets as CSV files.

The generated CSV files are later used during the data cleaning and database creation phases.

Why Python Was Used?

Python was selected because it provides a simple and efficient way to automate repetitive tasks.

Using Python allows:

- Automatic generation of hundreds of records.
- Easy modification of dataset size.
- Fast execution.
- Easy integration with external libraries.
- Simple CSV file creation.

Python also makes the project reusable because the datasets can be regenerated whenever required.

Why Faker Library Was Used?

The **Faker** library was used to generate realistic customer information.

Instead of manually creating names and email addresses, Faker automatically generates synthetic data that closely resembles real-world customer information.

Examples of generated values include:

- Customer Names
- Email Addresses
- Registration Dates

Using Faker provides several advantages:

- Creates realistic datasets.
- Generates unique records automatically.
- Eliminates the need for real customer information.
- Protects privacy.
- Makes the project suitable for testing and learning purposes.

Since the generated information is completely synthetic, no personal or confidential data is used in the project.

Why Random Module Was Used?

The Python **random** module was used to generate values that naturally vary in an e-commerce environment.

Examples include:

- Product selection
- Order quantity
- Unit price
- Discount percentage
- Customer type
- Order status
- Region code

Randomization ensures that every generated dataset contains different values, making the project more realistic and suitable for SQL analysis.

Generated Datasets

The data generation program creates the following datasets.

Customers Dataset

This dataset stores customer-related information.

The generated attributes include:

- Customer ID
- Customer Name
- Email Address
- Registration Date
- Customer Type

This dataset represents the customer master table of the e-commerce platform.

Products Dataset

This dataset stores product-related information.

Each record contains:

- Product ID
- Product Name
- Category
- Subcategory
- Cost Price

The product dataset is later used for category-wise analysis and revenue calculations.

Orders Dataset

The orders dataset represents customer purchases.

Each order contains:

- Order ID
- Customer ID
- Order Date
- Order Status
- Region Code

The order status simulates different business situations such as Delivered, Returned, Cancelled, Shipped, and Placed orders.

Order Items Dataset

The order items dataset stores the individual products included in each order.

Each record contains:

- Item ID
- Order ID
- Product ID
- Quantity
- Unit Price
- Discount Percentage

This dataset is mainly used for revenue calculations and advanced SQL analysis.

Why Were Intentional Errors Added?

Real-world business datasets rarely contain perfectly clean data.

To simulate practical data analysis scenarios, several intentional errors were introduced during dataset generation.

These include:

- Missing Customer IDs
- Invalid Email Addresses
- Incorrect Date Formats
- Negative Quantities

Adding these errors makes the next phase of the project more realistic because the data cleaning process demonstrates how such issues are identified and corrected before analysis.

Output

After successfully executing the program, the following files were generated.

```
Week-8/  
└─ data/  
    └─ raw/  
        ├── customers.csv  
        ├── products.csv  
        ├── orders.csv  
        └─ order_items.csv
```

These files contain the raw datasets that are used in the subsequent phases of the project.

Observation

The data generation phase completed successfully.

The Python program automatically created realistic customer, product, order, and transaction datasets without requiring manual data entry.

The generated datasets closely resemble real-world e-commerce data while intentionally including data quality issues for learning purposes.

These datasets provide the required input for the Data Cleaning phase, where they are validated and corrected before being stored in the SQLite database.

Phase 3 – Data Cleaning

Purpose

The data generated in the previous phase intentionally contains several inconsistencies to simulate real-world business scenarios. Before storing the data in a database and performing SQL analysis, it is important to ensure that the data is accurate, consistent, and reliable.

The main objective of this phase is to identify data quality issues and correct them using Python. Performing data cleaning before analysis helps improve the accuracy of SQL queries and ensures that business insights are based on valid information.

This phase demonstrates one of the most important steps in the data analysis pipeline because poor-quality data can lead to incorrect analysis and business decisions.

Source Code

The complete implementation of the data cleaning process is available in the following file.

```
Week-8/  
└─ python/  
    └─ clean_data.py
```

The cleaned datasets generated during this phase are stored in:

```
Week-8/  
└─ data/  
    └─ cleaned/
```

A summary of the identified and corrected issues is stored in:

```
Week-8/  
└─ data/  
    └─ cleaned/  
        └─ issues_report.csv
```

Implementation

The data cleaning program performs multiple validation and correction steps before the data is loaded into the SQLite database.

The implementation includes:

- Reading the raw datasets.
- Detecting missing values.
- Correcting incorrect date formats.
- Standardizing product names.
- Validating customer email addresses.
- Handling missing customer IDs.
- Checking referential integrity.
- Generating a summary report of all detected issues.

After completing all validation steps, the cleaned datasets are exported into the **data/cleaned** directory.

Why Pandas Was Used?

The **Pandas** library was used because it provides powerful tools for reading, cleaning, and manipulating tabular data.

Pandas made the implementation much easier by allowing operations such as:

- Reading CSV files.
- Updating incorrect values.
- Detecting missing records.
- Filtering rows.
- Exporting cleaned datasets.
- Performing validation checks.

Without Pandas, many of these operations would require significantly more code.

Cleaning Operations Performed

1. Date Format Correction

Some order records intentionally contained incorrect date formats.

The program checks every date and converts invalid formats into a standard format that can be used consistently throughout the project.

Using a standard date format is important because SQL functions such as grouping by month or year require valid dates.

2. Missing Customer ID Handling

Some order records intentionally contained missing Customer IDs.

These missing values were replaced with the value **"UNKNOWN"** so that the missing records remain identifiable instead of being completely removed.

This approach preserves the dataset while clearly indicating incomplete information.

3. Product Name Standardization

Some product names contained unnecessary spaces or inconsistent capitalization.

The cleaning program:

- Removes extra spaces.
- Converts names into Title Case.

Standardizing product names improves consistency and prevents duplicate-looking values during analysis.

4. Email Validation

Some customer email addresses were intentionally created with invalid formats.

Regular Expressions (Regex) were used to validate every email address.

Whenever an invalid email was detected, it was replaced with a placeholder email address.

This ensures that every customer record contains a valid email format.

5. Referential Integrity Check

Referential integrity was verified across all related datasets.

The program checked whether:

- Every Customer ID in the Orders table exists in the Customers table.
- Every Product ID in the Order Items table exists in the Products table.
- Every Order ID in the Order Items table exists in the Orders table.

This validation ensures that relationships between tables remain consistent before loading the data into the database.

6. Issues Report Generation

After completing the cleaning process, a separate report was generated to summarize all detected and corrected issues.

The report includes information such as:

- Missing Customer IDs
- Incorrect Date Formats
- Product Name Corrections
- Invalid Email Corrections

Generating this report makes it easier to verify the effectiveness of the cleaning process.

Why Data Cleaning Is Important?

Real-world datasets are rarely perfect.

Without proper cleaning, datasets may contain:

- Missing values
- Incorrect formats
- Invalid records
- Inconsistent values
- Broken relationships

Such issues can produce incorrect SQL query results and misleading business insights.

Performing data cleaning before database creation improves:

- Data Quality
 - Data Consistency
 - Query Accuracy
 - Database Reliability
 - Business Decision Making
-

Output

After successfully executing **clean_data.py**, the following cleaned datasets were generated.

```
Week-8/  
└─ data/  
    └─ cleaned/  
        ├── cleaned_customers.csv  
        ├── cleaned_products.csv  
        ├── cleaned_orders.csv  
        └─ issues_report.csv
```

These cleaned datasets are later used to create the SQLite database.

Observation

The data cleaning phase completed successfully.

All major data quality issues introduced during data generation were identified and corrected. The cleaned datasets became more consistent, reliable, and suitable for database storage and SQL analysis.

The generated issues report also provides a clear summary of all corrections performed during this phase, making the cleaning process transparent and easy to verify.

The successful completion of this phase ensured that the SQLite database was created using validated and high-quality data.

Phase 4 – SQLite Database Creation

Purpose

After cleaning and validating the generated datasets, the next step is to store them in a structured database.

Instead of directly performing SQL analysis on CSV files, the cleaned data is loaded into a relational SQLite database. Storing the data in a database makes it easier to manage relationships between different datasets and execute SQL queries efficiently.

The main objective of this phase is to organize the cleaned datasets into multiple related tables so that meaningful business analysis can be performed using SQL.

This phase acts as a bridge between data preprocessing and SQL analysis.

Source Code

The complete implementation of the database creation process is available in the following file.

```
Week-8/  
└─ python/  
    └─ database.py
```

The SQLite database created during this phase is stored in:

```
Week-8/  
└─ database/  
    └─ ecommerce.db
```

Implementation

The database creation program performs the following operations:

- Establishes a connection with the SQLite database.
- Creates the required database tables.
- Reads the cleaned CSV datasets.
- Loads the cleaned data into the database tables.
- Verifies that all records are inserted successfully.
- Closes the database connection after completion.

The entire database creation process is fully automated using Python.

Why SQLite Was Used?

SQLite was selected because it is a lightweight relational database management system that does not require a separate database server.

Some advantages of SQLite are:

- Easy to install and use.
- No server configuration required.
- Stores the complete database in a single file.
- Fast for small and medium-sized projects.
- Fully supports SQL queries.
- Easily integrates with Python.

Because of these features, SQLite is widely used for learning database concepts, academic projects, and prototype applications.

Why sqlite3 Module Was Used?

Python provides the **sqlite3** module as a built-in library for working with SQLite databases.

The sqlite3 module was used because it allows Python programs to:

- Connect to SQLite databases.
- Create database tables.
- Execute SQL commands.
- Insert records.
- Retrieve data.
- Close database connections safely.

Since sqlite3 is included with Python, no additional installation is required.

Why a Relational Database Was Used?

The generated data consists of multiple related entities such as customers, products, orders, and order items.

A relational database is suitable because it organizes data into separate tables while maintaining relationships between them.

Using separate tables provides several advantages:

- Reduces data duplication.
 - Improves data consistency.
 - Simplifies SQL queries.
 - Maintains relationships between datasets.
 - Supports efficient business analysis.
-

Database Tables

The following tables were created during this phase.

Customers Table

Stores customer-related information.

Contains details such as:

- Customer ID
- Customer Name
- Email Address
- Registration Date
- Customer Type

Products Table

Stores product-related information.

Contains:

- Product ID
 - Product Name
 - Category
 - Subcategory
 - Cost Price
-

Orders Table

Stores order-level information.

Contains:

- Order ID
- Customer ID
- Order Date
- Status
- Region Code

Each order is associated with a customer.

Order Items Table

Stores individual products purchased in each order.

Contains:

- Item ID
- Order ID
- Product ID
- Quantity
- Unit Price
- Discount Percentage

This table connects orders with products and is mainly used for revenue calculations.

Database Verification

After loading all datasets into the database, record verification was performed.

The verification process checks whether all records from the cleaned CSV files were successfully inserted into their respective tables.

This helps ensure that no data is lost during the database loading process.

Output

After successfully executing **database.py**, the following database file was created.

Week-8/

```
└─ database/  
    └─ ecommerce.db
```

The SQLite database contains the following tables:

- customers
- products
- orders
- order_items

These tables are later used during SQL analysis and Python-SQL integration.

Observation

The SQLite database was created successfully.

All cleaned datasets were loaded into their respective tables without any data loss.

Database verification confirmed that every table contained the expected number of records.

The successful completion of this phase provided a structured and reliable database for executing SQL queries and performing business analysis.

Phase 5 – SQL Analysis

Purpose

After successfully creating the SQLite database, the next phase of the project was to perform SQL-based business analysis.

The primary objective of this phase was to transform raw transactional data into meaningful business insights. SQL was used to retrieve, summarize, compare, and analyze the stored information to answer various business-related questions.

Instead of manually inspecting thousands of records, SQL queries were executed to generate useful reports such as revenue analysis, customer analysis, product performance, sales trends, customer segmentation, and retention analysis.

This phase demonstrates how SQL is used in real-world organizations to support business decision-making.

Source Code

All SQL queries developed during this phase are available in:

```
Week-8/  
└─ sql/  
    └─ basic_queries.sql
```

The execution screenshots of all SQL queries are stored in:

```
Week-8/  
└─ screenshots/  
    └─ sql_queries/
```

Implementation

The SQL analysis was performed on the SQLite database created in the previous phase.

The queries were executed using SQLite Explorer inside Visual Studio Code.

The implementation includes multiple categories of SQL operations such as:

- Data Retrieval
- Aggregation
- Filtering
- Sorting
- Joining Multiple Tables
- Window Functions
- Common Table Expressions (CTEs)
- Customer Segmentation
- Cohort Analysis
- Business Performance Analysis

Each query was executed successfully and its output was verified before moving to the next query.

Why SQL Was Used?

SQL (Structured Query Language) is the standard language used for managing and analyzing relational databases.

SQL allows users to:

- Retrieve specific information from databases.
- Filter unwanted records.
- Perform calculations.
- Generate reports.
- Analyze business performance.
- Identify trends and patterns.
- Support data-driven decision making.

Instead of processing CSV files manually, SQL provides a much faster and more efficient approach for analyzing structured data.

Types of SQL Queries Performed

The SQL implementation was divided into three major categories.

Basic SQL Queries

The basic queries focused on understanding the overall business performance.

These queries were used to:

- Calculate revenue generated by different product categories.
- Identify the top customers based on total spending.
- Analyze monthly order trends.

These analyses provide a high-level overview of sales performance.

Intermediate SQL Queries

Intermediate-level queries were developed to analyze customer and product behavior.

The analysis included:

- Customers whose orders were never delivered.
- Products with more returns than successful purchases.
- Return rate calculation for different product categories.

These queries help identify operational issues and improve customer satisfaction.

Advanced SQL Queries

The advanced section demonstrates analytical SQL concepts commonly used in real-world business environments.

Advanced SQL techniques implemented include:

- Running Total Calculation
- Product Ranking using DENSE_RANK()
- Revenue Comparison using LAG()
- Customer Segmentation using Multiple CTEs
- Customer Segmentation using NTILE()
- Year-over-Year Revenue Analysis
- First and Last Purchase Analysis
- Cumulative Distribution Analysis
- Cohort Analysis
- Self Join with Window Functions

These techniques help organizations analyze customer behavior, revenue growth, retention patterns, and long-term business performance.

Why Window Functions Were Used?

Window Functions allow calculations across multiple rows without losing individual row information.

Unlike GROUP BY, Window Functions preserve detailed records while simultaneously performing calculations.

The project uses several Window Functions including:

- DENSE_RANK()
- LAG()
- FIRST_VALUE()
- LAST_VALUE()
- NTILE()
- CUME_DIST()
- ROW_NUMBER()

These functions were mainly used for ranking, comparison, running totals, customer segmentation, and trend analysis.

Why Common Table Expressions (CTEs) Were Used?

Common Table Expressions (CTEs) improve the readability and organization of complex SQL queries.

Instead of writing long nested queries, CTEs divide complex logic into smaller and easier-to-understand sections.

Using CTEs provides several advantages:

- Improves query readability.
- Simplifies debugging.
- Makes SQL easier to maintain.
- Breaks complex calculations into logical steps.

The project uses CTEs to perform customer segmentation and revenue analysis.

Why Customer Segmentation Was Performed?

Not every customer contributes equally to business revenue.

Customer segmentation helps classify customers according to their purchasing behaviour and spending patterns.

This analysis helps businesses:

- Identify valuable customers.
- Design personalized marketing campaigns.
- Improve customer retention.
- Allocate business resources effectively.

The project performs customer segmentation using SQL ranking functions and NTILE().

Why Cohort Analysis Was Performed?

Cohort Analysis groups customers based on the time of their first purchase and tracks their activity over time.

Instead of analyzing all customers together, customers are grouped into cohorts to study long-term behaviour.

This analysis helps businesses understand:

- Customer retention.
- Repeat purchase behaviour.
- Customer engagement over time.
- Long-term business growth.

Cohort analysis is widely used in subscription services, e-commerce platforms, and digital businesses.

Business Insights Generated

The SQL analysis helped generate several meaningful business insights, including:

- Revenue generated by each product category.
- Identification of top-performing customers.
- Monthly order trends.
- Product return behaviour.
- Category-wise return rates.
- Customer purchase history.
- Revenue growth trends.
- Customer lifetime value.
- Customer segmentation.
- Customer retention analysis.

These insights help organizations make informed business decisions using data.

Output

The SQL analysis generated multiple business reports through SQL queries.

The SQL scripts are available in:

```
Week-8/  
└─ sql/  
    └─ basic_queries.sql
```

The execution outputs and screenshots are available in:

```
Week-8/  
└─ screenshots/  
    └─ sql_queries/
```

Each query was executed successfully on the SQLite database and verified before proceeding to the next analysis.

Observation

The SQL analysis phase successfully demonstrated how relational databases can be used to solve practical business problems.

Both basic and advanced SQL concepts were implemented to extract meaningful insights from the generated e-commerce dataset.

The implementation of joins, aggregation, window functions, Common Table Expressions, ranking functions, customer segmentation, and cohort analysis provided hands-on experience with real-world SQL techniques.

This phase significantly improved practical understanding of SQL and demonstrated its importance in business reporting, performance analysis, and data-driven decision making.

Phase 6 – Python and SQLite Integration

Purpose

After successfully generating the datasets, cleaning the data, creating the SQLite database, and performing SQL analysis, the final implementation phase of this project was to integrate Python with SQLite.

The objective of this phase is to demonstrate how SQL queries can be executed directly from a Python program instead of running them manually inside a database management tool.

Python-SQL integration is widely used in real-world applications where reports, dashboards, and automated data analysis are generated programmatically.

This phase demonstrates how Python can communicate with a relational database, execute SQL queries, retrieve the results, and display them in a structured format.

Source Code

The complete implementation of Python and SQLite integration is available in the following file.

```
Week-8/  
└─ python/  
    └─ run_queries.py
```

The database used during this phase is:

```
Week-8/  
└─ database/  
    └─ ecommerce.db
```

The execution screenshots are available in:

```
Week-8/  
└─ screenshots/  
    └─ integration/
```

Implementation

The integration process was implemented using Python.

The program performs the following steps:

1. Connects to the SQLite database.
2. Executes SQL queries.
3. Retrieves the query results.
4. Stores the results in a Pandas DataFrame.
5. Displays the output in a tabular format.
6. Closes the database connection safely.

The entire process is fully automated and requires minimal manual effort.

Why sqlite3 Module Was Used?

The **sqlite3** module is Python's built-in library for working with SQLite databases.

It was used because it provides a simple interface for database operations without requiring any additional installation.

The sqlite3 module allows Python to:

- Establish a connection with SQLite.
- Execute SQL queries.
- Read query results.
- Insert or update records.
- Close the database connection safely.

Since SQLite is lightweight and serverless, sqlite3 provides a fast and efficient way to interact with the database.

Why Pandas Was Used?

The **Pandas** library was used to retrieve and display SQL query results.

Instead of reading each database row individually, Pandas loads the complete query result into a DataFrame.

Using Pandas provides several advantages:

- Displays query results in a clean tabular format.
- Simplifies further data analysis.
- Makes the output easier to read.
- Supports exporting results into CSV or Excel if required.

Because of these features, Pandas is one of the most widely used libraries in data analysis projects.

Why Python and SQL Integration Is Important?

Modern organizations rarely execute SQL queries manually every day.

Instead, SQL queries are executed automatically through programming languages such as Python.

Python-SQL integration is commonly used for:

- Business reporting.
- Dashboard generation.
- Automated data analysis.
- Scheduled report generation.
- Machine Learning pipelines.
- Data Engineering workflows.

This integration helps automate repetitive tasks and improves overall productivity.

Output

The program successfully connected to the SQLite database and executed SQL queries.

The query results were displayed using Pandas DataFrames in the terminal.

The successful execution confirmed that Python and SQLite were integrated correctly.

Observation

The Python and SQLite integration phase completed successfully.

The database connection was established without errors, SQL queries executed correctly, and the results were displayed in a structured tabular format.

This phase demonstrated how Python can automate SQL execution and interact with relational databases in real-world applications.

The successful integration completed the end-to-end workflow of the project, starting from data generation and ending with automated SQL execution through Python.

Phase 7 – Project Summary and Conclusion

Challenges Faced

During the development of this project, several practical challenges were encountered. These challenges provided an opportunity to understand real-world data processing and database management more effectively.

Some of the major challenges faced during the project are listed below.

1. Generating Realistic Data

Since no real e-commerce dataset was provided, the complete dataset had to be generated from scratch.

Creating realistic customer records, products, orders, and transaction details while maintaining relationships between different datasets required careful planning.

The Faker library and Python's random module were used to overcome this challenge.

2. Maintaining Relationships Between Tables

The project consists of multiple related datasets such as customers, orders, products, and order items.

While generating the datasets, it was important to ensure that every Order ID, Customer ID, and Product ID correctly referenced the appropriate records.

Referential integrity checks were implemented during the data cleaning phase to validate these relationships.

3. Handling Data Quality Issues

Several intentional errors were introduced during dataset generation to simulate real-world business scenarios.

Examples include:

- Missing Customer IDs
- Invalid Email Addresses
- Incorrect Date Formats
- Negative Quantities

These issues were identified and corrected using Python before loading the data into the database.

4. Writing Advanced SQL Queries

Basic SQL queries were relatively straightforward, but implementing advanced SQL concepts such as Window Functions, Common Table Expressions (CTEs), Customer Segmentation, Cohort Analysis, and Year-over-Year analysis required careful understanding of SQL syntax and query logic.

Testing and verifying each query helped ensure correct results.

5. Python and SQLite Integration

Another challenge was integrating Python with SQLite and displaying SQL query results using Pandas.

This required understanding how Python communicates with relational databases and how query results can be processed programmatically.

Learning Outcomes

This project provided practical experience in multiple areas of data analysis and database management.

The following concepts were learned and implemented successfully:

Python Programming

- File handling
 - CSV processing
 - Functions
 - Modular programming
 - Library usage
-

Data Generation

- Synthetic dataset generation
 - Using the Faker library
 - Randomized transaction generation
 - Simulating real-world business data
-

Data Cleaning

- Handling missing values
- Data validation

- Email validation using Regular Expressions
 - Date format correction
 - Product name standardization
 - Referential integrity validation
-

Database Management

- SQLite database creation
 - Table creation
 - Data loading
 - Record verification
 - Database connectivity
-

SQL Concepts

The project helped strengthen practical knowledge of SQL, including:

- SELECT
 - WHERE
 - GROUP BY
 - ORDER BY
 - Aggregate Functions
 - INNER JOIN
 - Window Functions
 - DENSE_RANK()
 - ROW_NUMBER()
 - LAG()
 - FIRST_VALUE()
 - LAST_VALUE()
 - NTILE()
 - CUME_DIST()
 - Common Table Expressions (CTEs)
 - Customer Segmentation
 - Cohort Analysis
 - Year-over-Year Analysis
-

Python and SQL Integration

The project also demonstrated how Python can interact with relational databases.

The following concepts were implemented:

- SQLite Connection
- SQL Execution through Python
- Reading Query Results

- Displaying Results using Pandas DataFrames
-

Future Improvements

Although the project successfully meets the assignment requirements, several additional improvements can be made in the future.

Possible enhancements include:

- Using a larger dataset for analysis.
- Connecting the project to MySQL or PostgreSQL.
- Creating interactive dashboards using Power BI or Tableau.
- Building a web interface using Flask or Django.
- Automating report generation.
- Performing predictive analysis using Machine Learning.

These improvements would make the project more suitable for real-world enterprise applications.

Conclusion

This project successfully demonstrates an end-to-end data analysis workflow using Python, SQLite, and SQL.

The workflow began with generating realistic e-commerce datasets, followed by data cleaning and validation to improve data quality. The cleaned datasets were then stored in a SQLite database, where SQL queries were executed to analyze customer behaviour, product performance, revenue trends, return analysis, customer segmentation, and business growth.

Advanced SQL concepts such as Window Functions, Common Table Expressions (CTEs), Customer Segmentation, Cohort Analysis, and Year-over-Year analysis were successfully implemented to solve business-oriented analytical problems.

Finally, Python was integrated with SQLite to automate SQL execution and display query results programmatically using Pandas.

Overall, this project provided practical experience in data generation, data preprocessing, relational database management, SQL programming, and Python-SQL integration. It also demonstrated how structured data can be transformed into meaningful business insights, helping organizations make informed and data-driven decisions.

References

The following resources were used during the development of this project:

- [Python Official Documentation](#)
 - [Pandas Documentation](#)
 - [Faker Documentation](#)
 - [SQLite Documentation](#)
 - [SQL Language Reference](#)
 - [Celebal Summer Internship Week 8 Assignment PDF](#)
-